Конвертор из MathML у LaTeX

Пројекат из предмета Лексичка анализа

Студент: Јована Радисављевић
Асистент: Стефан Миленковић
Математички факултет, Београд
Март 2026.

Увод

Лексичка анализа представља прву фазу у процесу превођења програмских језика - њен задатак је да улазни текст разложи на низ токена, при чему сваки токен представља најмању смислену јединицу. У овом пројекту, лексичка анализа се примењује на MathML документе: улазни XML текст се токенизује помоћу регуларних израза, а добијени токени се даље користе за генерисање одговарајућег LaTeX кода.

Историјски гледано, потреба за прецизним представљањем математичких формула у дигиталним документима постојала је још од првих покушаја дигитализације научних радова. MathML (Mathematical Markup Language) омогућава структурирано кодирање математичких израза, док LaTeX пружа естетску и стандардизовану презентацију за академске публикације.

Изабрани примери у овом раду демонстрирају могућности конвертора. Ови примери садрже различите математичке конструкције: разломке, степене, корене, суме, интеграле, матрице и ограђене изразе, што омогућава демонстрацију функционалности конвертора кроз више типова MathML елемената.

1 Python имплементација конвертора

Регуларни изрази у лексеру

Лексер препознаје следеће класе токена помоћу регуларних израза:

- `OTVARAJUCI_TAG`: `<[a-zA-Z]+[^]*>` - препознаје отварајући XML таг са опционим атрибутима
- `ZATVARAJUCI_TAG`: `</[a-zA-Z]+>` - препознаје затварајући XML таг
- `ENTITET`: `&[a-zA-Z]+;` - препознаје MathML ентитете попут `∑`, `π` и `→`
- `BROJ`: `\d+` - препознаје целобројни садржај унутар `<mn>`
- `IDENT`: `[a-zA-Z]+` - препознаје идентификаторе унутар `<mi>`
- `OPERATOR`: `[+\-*/=!]` - препознаје математичке операторе
- `SIMBOL`: `[()[{}]]` - препознаје заграде различитих врста
- `PRAZNO`: `\s+` - препознаје белине (размаци, табулатори, нови редови); токени ове класе се одбацују

Свако правило се примењује редом - бира се прво правило чији регуларни израз покрива позицију у улазном тексту. Захваљујући томе, вишезначни низови као што је `∑` се исправно препознају као ENTITET, а не као низ засебних токена.

```
import re, sys

TAG_MAP = {
    "msqrt": ("\\sqrt{", "}"),
    "mtable": ("\\begin{bmatrix}", "\\end{bmatrix}"),
    "mtd": ("", "␣&␣"),
    "mtr": ("", "␣\\\\␣"),
```

```
    "msin": ("\\sin", ""),
    "mcos": ("\\cos", "")
}

ENTITET_MAP = {
    "&sum;": "\\sum",
    "&int;": "\\int",
    "&pi;": "\\pi",
    "&theta;": "\\theta",
    "&rarr;": "\\rightarrow",
    "&times;": "\\times",
    "&sdot;": "\\cdot",
    "&ApplyFunction;": "␣",
    "&infin;": "\\infty",
    "&alpha;": "\\alpha",
    "&beta;": "\\beta",
}

FUNC_MAP = {
    "sin": "\\sin", "cos": "\\cos", "tan": "\\tan", "cot": "\\cot",
    "log": "\\log", "ln": "\\ln", "exp": "\\exp", "inf": "\\inf",
    "lim": "\\lim"
}

PRAVILA = [
    ("OTVARAJUCI_TAG", r"<[a-zA-Z]+[^>]*>"),
    ("ZATVARAJUCI_TAG", r"</[a-zA-Z]+>"),
    ("ENTITET", r"&[a-zA-Z]+;"),
    ("BROJ", r"\d+"),
    ("IDENT", r"[a-zA-Z]+"),
    ("OPERATOR", r"[+\-*/=!]"),
    ("PRAZNO", r"\s+"),
    ("SIMBOL", r"[(){}[\]]")
]

def tokenizuj(ulazni_tekst):
    kompajlirana = [(tip, re.compile(pat)) for tip, pat in PRAVILA]
    pozicija = 0
    tokeni = []
    while pozicija < len(ulazni_tekst):
        pronadjen = False
        for tip, pattern in kompajlirana:
            m = pattern.match(ulazni_tekst, pozicija)
            if m:
                pronadjen = True
                if tip != "PRAZNO":
                    tokeni.append((tip, m.group()))
                pozicija = m.end()
                break
```

```
        if not pronadjen:
            tokeni.append(("NEPOZNATO", ulazni_tekst[pozicija]))
            pozicija += 1
    return tokeni

def ispisi_tokene(tokeni):
    print(f"{'Br.':<5} {'Tip':<20} {'Vrednost'}")
    print("-" * 50)
    for i, (tip, vrednost) in enumerate(tokeni, 1):
        print(f"{i:<5} {tip:<20} {vrednost}")

def izvuci_ime_taga(tag_str):
    m = re.match(r"<([a-zA-Z]+)", tag_str)
    return m.group(1) if m else ""

def izvuci_atribut(tag_str, naziv):
    m = re.search(rf'{naziv}="([^"]*)"', tag_str)
    return m.group(1) if m else None
def prevedi_tag(tag, deca, atributi):
    if tag == "mfrac":
        if len(deca) >= 2:
            return f"\\frac{{{deca[0]}}}{{{deca[1]}}}"
        return deca[0] if deca else ""
    elif tag == "msqrt":
        return f"\\sqrt{{{deca[0]}}}" if deca else ""
    elif tag == "mtable":
        return f"\\begin{{matrix}}{' \\\\ '.join(deca)}\\end{{matrix}}"
    elif tag == "mtr":
        return " & ".join(deca)
    elif tag == "mtd":
        return "".join(deca)
    elif tag == "mi":
        sadrzaj = "".join(deca).strip()
        return FUNC_MAP.get(sadrzaj, sadrzaj)
    elif tag in ("msin", "mcos"):
        return TAG_MAP[tag][0]
    elif tag in ("msub", "msup", "munder", "mover"):
        if len(deca) >= 2:
            if tag == "msub":
                return f"{deca[0]}_{{{deca[1]}}}"
            elif tag == "msup":
                return f"{deca[0]}^{{{deca[1]}}}"
            elif tag == "munder":
                return f"\\underset{{{deca[1]}}}{{{deca[0]}}}"
            elif tag == "mover":
                return f"\\overset{{{deca[1]}}}{{{deca[0]}}}"
        return deca[0] if deca else ""
    elif tag in ("msubsup", "munderover"):
        if len(deca) >= 3:
            return f"{deca[0]}_{{{deca[1]}}}^{{{deca[2]}}}"
```

```
        elif len(deca) == 2:
            return f"{deca[0]}_{{{deca[1]}}}"
        return deca[0] if deca else ""
    elif tag == "mfenced":
        otvorena = atributi.get("open", "(")
        zatvorena = atributi.get("close", ")")
        sadrzaj = "".join(deca)
        return f"\\left{otvorena}␣{sadrzaj}␣\\right{zatvorena}"
    else:
        return "".join(deca)

def konvertuj_rekurzivno(tokeni, pozicija):
    deca = []
    while pozicija < len(tokeni):
        tip, vrednost = tokeni[pozicija]
        if tip == 'ZATVARAJUCI_TAG':
            return deca, pozicija + 1
        elif tip == 'OTVARAJUCI_TAG':
            tag_ime = izvuci_ime_taga(vrednost)
            atributi = {}
            if tag_ime == "mfenced":
                otvorena = izvuci_atribut(vrednost, "open")
                zatvorena = izvuci_atribut(vrednost, "close")
                if otvorena: atributi["open"] = otvorena
                if zatvorena: atributi["close"] = zatvorena
            deca_trenutno, pozicija = konvertuj_rekurzivno(tokeni,
                pozicija+1)

            latex = prevedi_tag(tag_ime, deca_trenutno, atributi)
            deca.append(latex)
        elif tip == 'ENTITET':
            deca.append(ENTITET_MAP.get(vrednost, vrednost))
            pozicija+=1
        else:
            deca.append(vrednost)
            pozicija += 1
    return deca,pozicija

def konvertuj(tokeni):
    deca, _ = konvertuj_rekurzivno(tokeni, 0)
    return "".join(deca)

def napravi_latex_doc(formula):
    return ("\\documentclass{article}\n"
            "\\usepackage{amsmath}\n"
            "\\begin{document}\n"
            "\\[\n"
            + formula +
            "\n\\]\n"
            "\\end{document}")

```

```python

def konvertuj_fajl(ulazni_fajl, izlazni_fajl):
    with open(ulazni_fajl, "r", encoding="utf-8") as f:
        mathml_tekst = f.read()

    tokeni = tokenizuj(mathml_tekst)
    ispisi_tokene(tokeni)

    latex_tekst = konvertuj(tokeni)
    latex_doc = napravi_latex_doc(latex_tekst)

    with open(izlazni_fajl, "w", encoding="utf-8") as f:
        f.write(latex_doc)

    print("LaTeX formula:")
    print(latex_tekst)


if len(sys.argv) != 3:
    print("Greska: neispravno pozivanje programa")
    sys.exit(1)

ulazni_fajl  = sys.argv[1]
izlazni_fajl = sys.argv[2]

konvertuj_fajl(ulazni_fajl, izlazni_fajl)
```

Ограничења конвертора

Имплементирани конвертор обрађује подскуп MathML елемената. Нису подржани: угнеждени `mfenced` са произвољним атрибутима, `msubsup` са три независна детета различитих типова, MathML ентитети ван дефинисане листе, као и вишелинијски LaTeX излаз са поравнањем (`align`). Такође, претпоставља се да је улаз синтаксно исправан. Напомена: класа `MathMLKonverter` обједињује токенизацију, обраду стека и генерисање излазног кода, те представља прост генератор кода заснован на лексичкој анализи.

2 Примери улаза и излаза

Пример 1. комбинаторика

Улаз (MathML):

```
<math>
  <mrow>
    <mi>P</mi><mo>=</mo>
    <mfrac>
      <mrow><mn>26</mn><mo>!</mo></mrow>
      <mrow><mn>3</mn><mo>!</mo><mo>&sdot;</mo><mn>23</mn><mo>!</
         mo></mrow>
    </mfrac>
  </mrow>
</math>
```

Токени које производи лексер:

Бр.	Тип	Вредност	Бр.	Тип	Вредност
1	OTVARAJUCI_TAG	<math>	20	BROJ	3
2	OTVARAJUCI_TAG	<mrow>	21	ZATVARAJUCI_TAG	</mn>
3	OTVARAJUCI_TAG	<mi>	22	OTVARAJUCI_TAG	<mo>
4	IDENT	P	23	OPERATOR	!
5	ZATVARAJUCI_TAG	</mi>	24	ZATVARAJUCI_TAG	</mo>
6	OTVARAJUCI_TAG	<mo>	25	OTVARAJUCI_TAG	<mo>
7	OPERATOR	=	26	ENTITET	⋅
8	ZATVARAJUCI_TAG	</mo>	27	ZATVARAJUCI_TAG	</mo>
9	OTVARAJUCI_TAG	<mfrac>	28	OTVARAJUCI_TAG	<mn>
10	OTVARAJUCI_TAG	<mrow>	29	BROJ	23
11	OTVARAJUCI_TAG	<mn>	30	ZATVARAJUCI_TAG	</mn>
12	BROJ	26	31	OTVARAJUCI_TAG	<mo>
13	ZATVARAJUCI_TAG	</mn>	32	OPERATOR	!
14	OTVARAJUCI_TAG	<mo>	33	ZATVARAJUCI_TAG	</mo>
15	OPERATOR	!	34	ZATVARAJUCI_TAG	</mrow>
16	ZATVARAJUCI_TAG	</mo>	35	ZATVARAJUCI_TAG	</mfrac>
17	ZATVARAJUCI_TAG	</mrow>	36	ZATVARAJUCI_TAG	</mrow>
18	OTVARAJUCI_TAG	<mrow>	37	ZATVARAJUCI_TAG	</math>
19	OTVARAJUCI_TAG	<mn>			

Напомена: токени типа PRAZNO (белине) се одбацују током токенизације и не појављују се у табели.

Излаз (LaTeX):

$$P = \frac{26!}{3! \cdot 23!}$$

Пример 2. матрице

Улаз (MathML):

```
<math>
  <mrow>
    <mfenced open="[" close="]">
```

```
      <mtable>
        <mtr>
          <mtd><msub><mi>R</mi><mn>1</mn></msub></mtd>
          <mtd><msub><mi>R</mi><mn>2</mn></msub></mtd>
        </mtr>
        <mtr>
          <mtd><msub><mi>R</mi><mn>3</mn></msub></mtd>
          <mtd><msub><mi>R</mi><mn>4</mn></msub></mtd>
        </mtr>
      </mtable>
    </mfenced>
    <mo>&times;</mo>
    <mfenced open="[" close="]">
      <mtable>
        <mtr><mtd><msub><mi>x</mi><mn>1</mn></msub></mtd></mtr>
        <mtr><mtd><msub><mi>x</mi><mn>2</mn></msub></mtd></mtr>
      </mtable>
    </mfenced>
  </mrow>
</math>
```

Излаз (LaTeX):

$$\begin{bmatrix} R_1 & R_2 \\ R_3 & R_4 \end{bmatrix} \cdot \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

Улаз (MathML):

```
<math>
  <mrow>
    <msubsup>
      <mo>&sum;</mo>
      <mrow><mi>i</mi><mo>=</mo><mn>1</mn></mrow>
      <mn>n</mn>
    </msubsup>
    <msup><mi>i</mi><mn>2</mn></msup>
  </mrow>
</math>
```

Излаз (LaTeX):

$$\sum_{i=1}^{n} i^2$$

Пример 4. интеграли

Улаз (MathML):

```
<math>
  <mrow>
    <msubsup><mo>&int;</mo><mn>0</mn><mi>&pi;</mi></msubsup>
    <mrow><mi>sin</mi><mo>&ApplyFunction;</mo><mi>x</mi></mrow>
    <mrow><mi>d</mi><mi>x</mi></mrow>
  </mrow>
</math>
```

Излаз (LaTeX):

$$\int_0^\pi \sin x \, dx$$

Пример 5. вектори

Улаз (MathML):

```
<math>
  <mrow>
    <mover><mi>F</mi><mo>&rarr;</mo></mover>
    <mo>=</mo>
    <mrow><mi>cos</mi><mo>&ApplyFunction;</mo><mi>&theta;</mi></
       mrow>
    <mo>+</mo>
    <munderover>
      <mo>&sum;</mo>
      <mrow><mi>k</mi><mo>=</mo><mn>1</mn></mrow>
      <mi>n</mi>
    </munderover>
    <msup><mi>k</mi><mn>2</mn></msup>
  </mrow>
</math>
```

Излаз (LaTeX):

$$\vec{F} = \cos\theta + \sum_{k=1}^{n} k^2$$

Закључак

Предложени систем демонстрира пренос математичких израза из дигиталних докумената у формални академски LaTeX формат. Изабрани примери обухватају кључне математичке структуре као што су комбинаторика, интеграли, матрице, суме и функције. Такође показују примену различитих MathML елемената и омогућавају увид у сложене математичке операције у академским текстовима, што је значајно за студенте који желе да разумеју практичну примену лексичке анализе у обради математичких докумената.